

Week 7

Text Generation using LSTM

Aim of the Program:

The aim of this program is to develop a Text Generation system using a Long Short-Term Memory (LSTM) neural network in Python. The program trains the LSTM model on a given text dataset to learn the patterns, relationships, and sequence of words. After training, the model generates new text automatically based on a given seed text during execution time. The program also visualizes the training performance using accuracy and loss plots to analyze the learning behavior of the model..

Import libraries

1. `import numpy as np`
2. `import tensorflow as tf`
3. `import matplotlib.pyplot as plt`
4. `from tensorflow.keras.models import Sequential`
5. `from tensorflow.keras.layers import LSTM, Dense, Embedding`
6. `from tensorflow.keras.preprocessing.text import Tokenizer`
7. `from tensorflow.keras.preprocessing.sequence import pad_sequences`

- ❖ An **embedding layer** is a crucial neural network component that converts high-dimensional, sparse categorical data (like word indices) into low-dimensional, dense, and continuous vectors.
- ❖ These vectors are learned during model training, allowing semantically similar items to have similar vector representations.
- ❖ They are essential in NLP, recommendation systems, and computer vision to capture complex relationships.

Sample dataset

```
1. text_data = [  
2.     "Deep learning is powerful",  
3.     "Deep learning is interesting",  
4.     "deep learning is part of machine learning",  
5.     "artificial intelligence is the future",  
6.     "deep learning drives artificial intelligence",  
7.     "Subject Teaching - Dr.M.Narayanan"  
8. ]
```

Tokenization

1. `tokenizer = Tokenizer()`
2. `tokenizer.fit_on_texts(text_data)`
3. `total_words = len(tokenizer.word_index) + 1`

Why +1 ?

Index **0** is reserved for padding in neural networks.

❖ **This line creates a Tokenizer object from the Keras preprocessing library.**

Purpose:

- ❖ It is used to convert words into numbers (tokens).
- ❖ Neural networks cannot understand text directly, so words must be encoded as integers.
 - ❖ `Tokenizer()` → Creates tool to convert text to numbers
 - ❖ `fit_on_texts()` → Learns all words from dataset
 - ❖ `word_index` → Stores word-to-number mapping
 - ❖ `total_words` → Counts vocabulary size for the neural network

Create sequences

```
1. input_sequences = []
2. for line in text_data:
3.     token_list = tokenizer.texts_to_sequences([line])[0]
4.     for i in range(1, len(token_list)):
5.         n_gram_sequence = token_list[:i+1]
6.         input_sequences.append(n_gram_sequence)
```

- ❖ This code **converts each sentence into multiple progressive word sequences (n-grams)** so the LSTM model can learn to predict the next word in a sentence.
- ❖ **n-grams are continuous sequences of “n” words taken from a sentence.**

Ex. Deep learning is powerful

1-gram (Unigram) – single word

Deep

learning

is

powerful

2-gram (Bigram) – two words

Deep learning

learning is

is powerful

3-gram (Trigram) – three words

❖ Deep learning is
learning is powerful

Padding sequences

1. `max_sequence_len = max(len(seq) for seq in input_sequences)`
 2. `input_sequences = np.array(`
 3. `pad_sequences(input_sequences, maxlen=max_sequence_len, padding='pre')`
 4. `)`
- ❖ This code finds the longest sequence length and pads all other sequences with zeros at the beginning so that every input sequence has the same length for the LSTM model.

Split data

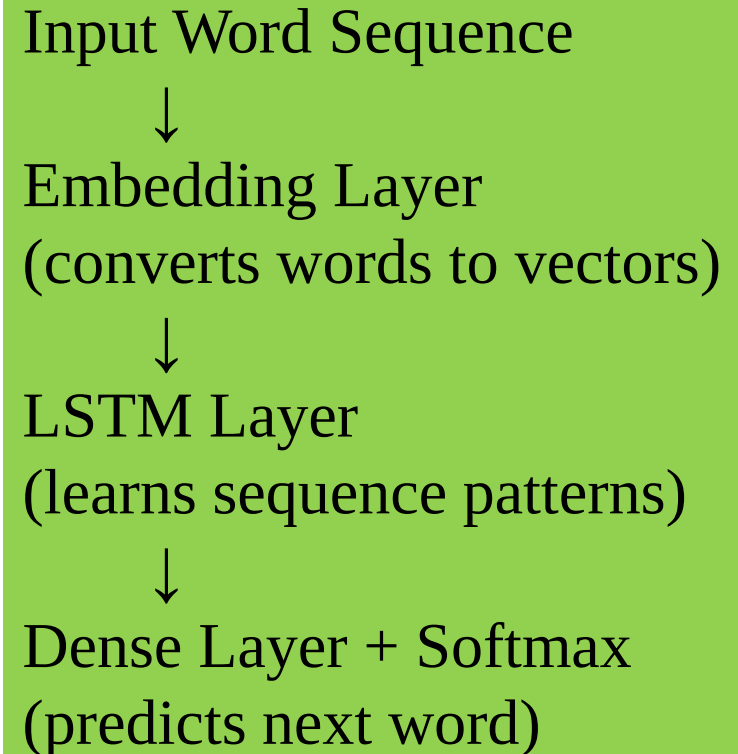
1. `X = input_sequences[:, :-1]`
2. `y = input_sequences[:, -1]`

Convert output to categorical

1. `y = tf.keras.utils.to_categorical(y, num_classes=total_words)`

Build LSTM Model

1. `model = Sequential()`
2. `model.add(Embedding(total_words, 10, input_length=max_sequence_len-1))`
3. `model.add(LSTM(100))`
4. `model.add(Dense(total_words, activation='softmax'))`
5. `model.compile(`
6. `loss='categorical_crossentropy',`
7. `optimizer='adam',`
8. `metrics=['accuracy']`
9. `)`



Train the model

1. `print("Training the LSTM model...\n")`
2. `history = model.fit(X, y, epochs=150, verbose=1)`

```
Training the LSTM model...
```

```
Epoch 1/150
```

```
1/1 [=====] - 4s 4s/step - loss: 2.9444 - accuracy: 0.0417
```

```
Epoch 2/150
```

```
1/1 [=====] - 0s 31ms/step - loss: 2.9408 - accuracy: 0.0833
```

```
Epoch 3/150
```

```
1/1 [=====] - 0s 34ms/step - loss: 2.9372 - accuracy: 0.2083
```

```
Epoch 4/150
```

```
1/1 [=====] - 0s 28ms/step - loss: 2.9335 - accuracy: 0.2083
```

```
Epoch 5/150
```

```
1/1 [=====] - 0s 23ms/step - loss: 2.9296 - accuracy: 0.2083
```

```
Epoch 144/150
```

```
1/1 [=====] - 0s 38ms/step - loss: 0.6338 - accuracy: 0.7917
```

```
Epoch 145/150
```

```
1/1 [=====] - 0s 31ms/step - loss: 0.6225 - accuracy: 0.7917
```

```
Epoch 146/150
```

```
1/1 [=====] - 0s 31ms/step - loss: 0.6109 - accuracy: 0.7917
```

```
Epoch 147/150
```

```
1/1 [=====] - 0s 28ms/step - loss: 0.6006 - accuracy: 0.7917
```

```
Epoch 148/150
```

```
1/1 [=====] - 0s 28ms/step - loss: 0.5916 - accuracy: 0.7917
```

```
Epoch 149/150
```

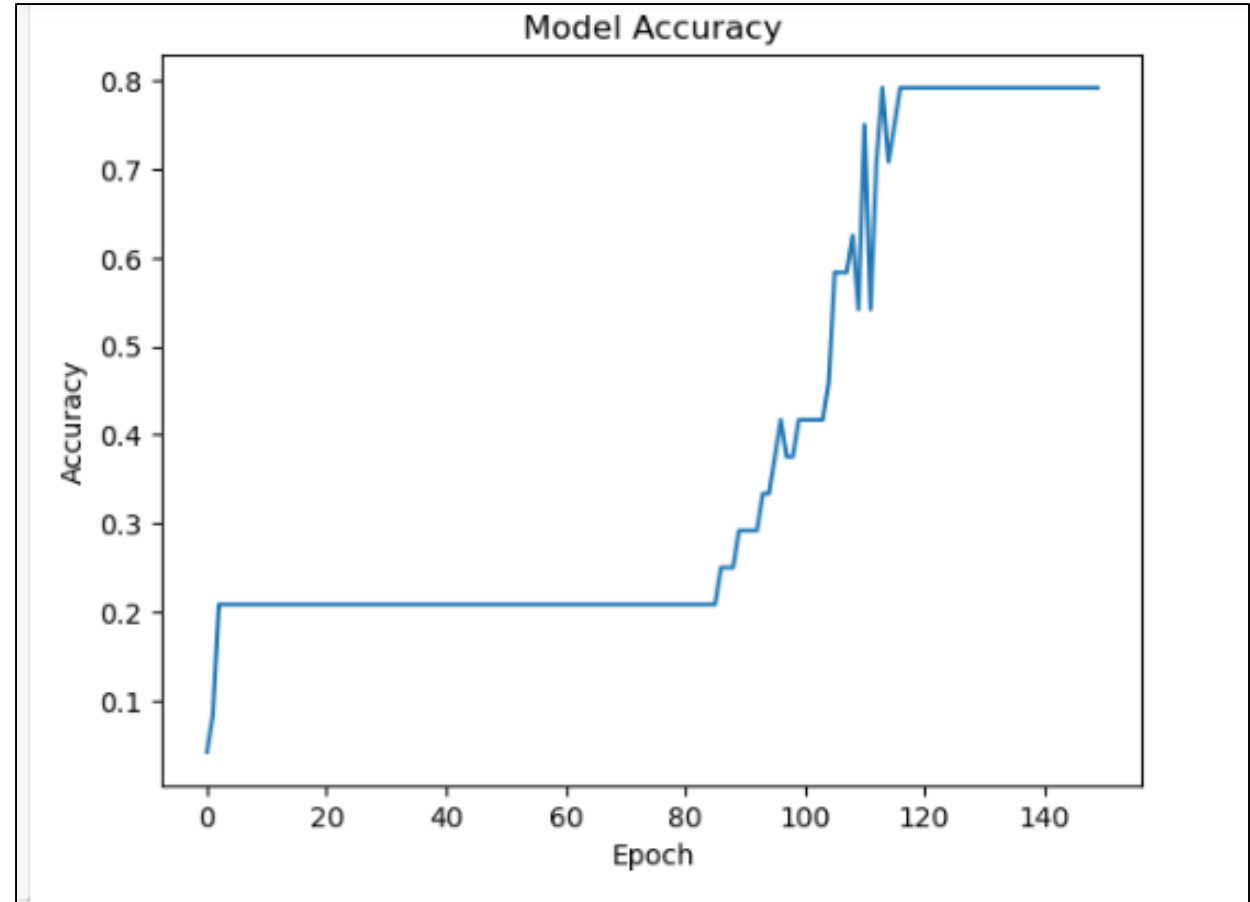
```
1/1 [=====] - 0s 30ms/step - loss: 0.5826 - accuracy: 0.7917
```

```
Epoch 150/150
```

```
1/1 [=====] - 0s 25ms/step - loss: 0.5730 - accuracy: 0.7917
```

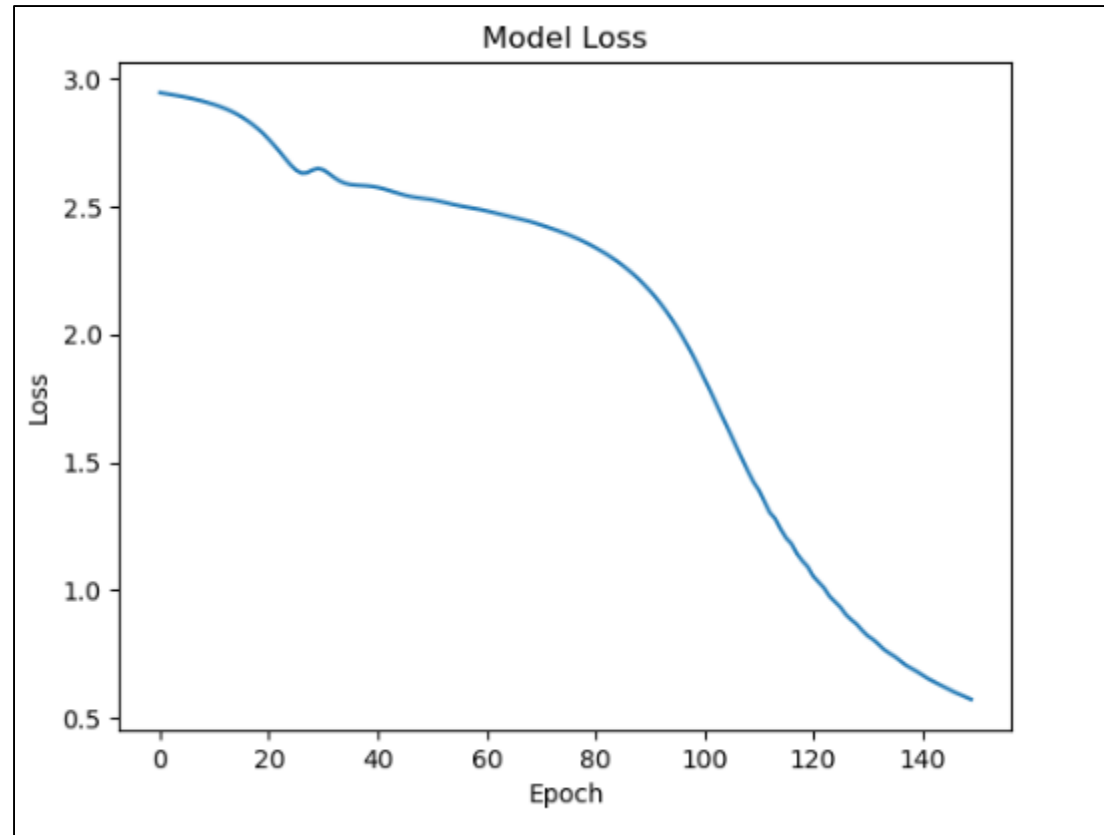
Plot Accuracy

1. `plt.figure()`
2. `plt.plot(history.history['accuracy'])`
3. `plt.title("Model Accuracy")`
4. `plt.xlabel("Epoch")`
5. `plt.ylabel("Accuracy")`
6. `plt.show()`



Plot Loss

1. `plt.figure()`
2. `plt.plot(history.history['loss'])`
3. `plt.title("Model Loss")`
4. `plt.xlabel("Epoch")`
5. `plt.ylabel("Loss")`
6. `plt.show()`



Function to generate text

```
1. def generate_text(seed_text, next_words):
2.     print("\nGenerating text during execution...\n")
3.     for i in range(next_words):
4.         token_list = tokenizer.texts_to_sequences([seed_text])[0]
5.         token_list = pad_sequences([token_list],
6.                                   maxlen=max_sequence_len-1,
7.                                   padding='pre')
8.         predicted = model.predict(token_list, verbose=0)
9.         predicted_index = np.argmax(predicted)
10.        output_word = ""
11.        for word, index in tokenizer.word_index.items():
12.            if index == predicted_index:
13.                output_word = word
14.                break
15.        seed_text += " " + output_word
16.        # Print word during execution
17.        print("Generated word:", output_word)
18.    return seed_text
```

- ❖ The **generate_text()** function is used to **generate new text** using the trained LSTM model.
- ❖ It takes **two inputs**: **seed_text**, which is the starting sentence, and **next_words**, which specifies how many new words the model should generate.
- ❖ First, the function prints a message indicating that text generation has started during execution.
- ❖ Then a loop runs for the number of words to be generated.
- ❖ Inside the loop, the seed text is converted into numerical tokens using the tokenizer, and the sequence is padded so that its length matches the input length expected by the LSTM model.
- ❖ The padded sequence is then given to the trained model to predict the probability of the next possible word.
- ❖ The index of the word with the highest probability is selected using `argmax`. This index is converted back to the corresponding word by searching in the tokenizer's word index dictionary. The predicted word is appended to the seed text, and it is also printed during execution so that the user can see each generated word step by step. This process repeats until the required number of words is generated, and finally the complete generated sentence is returned.

Seed text

1. seed = "Deep learning"

Generate text in runtime

1. generated_text = generate_text(seed, 7)
2. print("\n Final Generated Text:")
3. print(generated_text)

```
Generating text during execution...
```

```
Generated word: is
```

```
Generated word: interesting
```

```
Generated word: of
```

```
Generated word: machine
```

```
Generated word: learning
```

```
Generated word: learning
```

```
Generated word: learning
```

```
Final Generated Text:
```

```
Deep learning is interesting of machine learning learning learning
```

Conclusion:

The program successfully demonstrates how an LSTM-based deep learning model can be used for automatic text generation. By training on a small dataset, the model learns the sequence patterns of words and predicts the next possible word in a sentence. The generated text shows that LSTM networks are effective in handling sequential data and maintaining contextual information. The training accuracy and loss plots help in evaluating the performance of the model during learning. This approach can be extended to larger datasets for applications such as chatbots, automatic content generation, language modeling, and predictive text systems.